DD2482 Project Report

Singvalliyappa Velayutham (sinvel@kth.se) ·
Robert Jensen (robjen@kth.se)
Repositories:
web-site-backend
web-site-admin
website-frontend

1 System and goal

Our project is a portfolio website builder that we made before this course. You log in with Google, edit your portfolio in an admin panel and get your own public site on a subdomain. The code is split into three repositories: a NestJS backend that uses MongoDB Atlas, a React admin panel and a React public frontend (both built with Vite). Before the course there was no automation at all, so we built and deployed everything by hand. The goal of the project was not to add new features. Instead we wanted to build a DevOps pipeline around an app that already exists, so that every change is tested automatically, deployed automatically and runs on infrastructure that is written as code.

2 Architecture and processes

Figure 1 shows how the parts fit together. Code goes from GitHub to GitHub Actions, which tests it and then deploys it: the backend as a Docker image through GHCR to Render, and the two frontends to Vercel. The backend stores its data in MongoDB Atlas, and both Render and Atlas are set up with Terraform. Table 1 shows how we cover each requirement of the project.

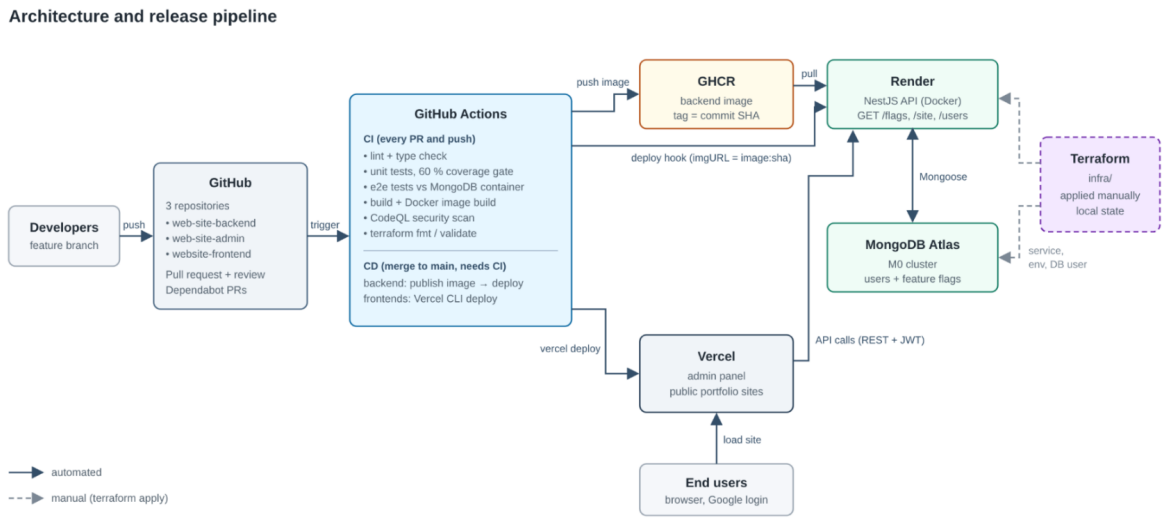


Fig 1 Architecture and release pipeline. Solid arrows are automated, dashed arrows are manual (terraform apply).

Required feature	How it is implemented
Development platform	GitHub. We work in feature branches and merge through pull requests. A ruleset on main requires a pull request and a passing CI check, and blocks changes that add a high severity CodeQL alert. We also use GitHub Actions, GitHub Container Registry (GHCR) and Environments
CI	Backend: lint, type check, 30 unit tests (the build fails if coverage drops below 60 %, right now it is 67 %), end-to-end tests against a MongoDB container, build, and a test build of the Docker image. Admin and frontend: lint, type check and build.
CD	Backend: when something is merged to main, CI builds a Docker image, pushes it to GHCR tagged with the commit SHA and tells Render to run exactly that image through a deploy hook. Frontends: deployed to Vercel with the Vercel CLI from GitHub Actions.
Infrastructure as Code	Terraform sets up the MongoDB Atlas cluster, the database user and network access, and the Render web service with its environment variables. CI runs terraform fmt and terraform validate when something in infra/ changes.
Quality and security automation	CodeQL scans all three repositories on every push, every pull request and once a week. Dependabot opens weekly pull requests with npm updates. GitHub secret scanning checks that no secrets are pushed to the repositories.
Feature flags	Flags are stored in MongoDB and the backend returns them on GET /flags. Our second portfolio template is hidden behind the template2 flag.

Table 1 How each project requirement is implemented.

3 Key design decisions

Deploy only what CI tested. Both Render and Vercel can deploy by themselves on every push, which would skip CI completely. We turned that off: Render does not auto-deploy, and Git deployments are disabled in vercel.json. Instead, a deploy job is used in the same workflow that waits for the tests to pass (`needs:`) and only runs on main. This means code that fails CI can never reach production.

Render does not build the code itself. CI builds one image and Render runs it, so the image we tested is the same one that runs in production. If something breaks we can roll back by deploying an older SHA, since all old images are still in GHCR.

The Render service and the free Atlas cluster already existed. A new Render service would get a new URL, and a free cluster cannot be replaced without losing the data. So we imported the existing resources into Terraform and added `prevent_destroy` on the cluster, so it cannot be deleted by mistake. Passwords that nobody needs to know, like the database password and the JWT secret, are generated by Terraform instead of being typed in by hand.

We tested services, controllers, login strategies and input validation, and mocked the database, so all tests run in a few seconds. We chose not to unit test modules, schemas and startup code (the remaining ∼33 %), because that code only connects things together and is already covered by the end-to-end test..

4 How the components interact

A normal workflow in backend looks like this: we open a pull request → CI runs lint, type check, unit tests with the coverage limit, end-to-end tests, build and Docker build, and CodeQL scans the code → the other person reviews and merges → on main, CI pushes → the deploy job calls Render's deploy hook with that image. The frontends work the same way but are deployed to Vercel. Pull requests from Dependabot go through the same pipeline, so dependency updates are tested before we merge them.

Feature flags let us deploy code without turning it on. When the backend starts, it creates the flags document if it does not exist (without overwriting values that are already there). The public frontend fetches `/flags` when the page loads. If the flag is off, or the request fails, it shows template 1 instead. This means we can deploy a new feature hidden and turn it on or off in the database without deploying again.

5 Limitations and trade-offs

- **Terraform is run by hand.** CI only checks that the Terraform code is valid. The state file contains secrets, so it is kept locally and not committed. If more people should be able to apply changes, we would need a remote backend like Terraform Cloud. The Render provider also cannot update free services, so some changes have to be done by hand.
- **Free plans affect the design.** Render's free plan has no fixed IP address, so Atlas has to accept connections from anywhere (0.0.0.0/0) and only the password protects it. The free service also goes to sleep, so the first request after a while is slow.
- **No check after deploy.** The pipeline does not check that the new version actually works

after it is deployed, and rollback is manual (deploy an older SHA, or revert the merge so the pipeline runs again). A next step would be a simple smoke test that calls `/flags` after each deploy.

- **Not everything is tested.** The frontends are only linted and built, they have no tests. The end-to-end test only checks one endpoint.
- **Three repositories.** Each repository has its own pipeline, so nothing tests the frontends against a new backend before both are deployed. If we change the API, we have to coordinate the changes by hand.
- **Simple feature flags.** Flags are changed directly in the database, they apply to everyone at once (no gradual rollout) and they are only read when the page loads.
- **Shared accounts.** The free plans on Render and Vercel do not allow team members, so we share one account. That makes it harder to see who did what.

AI usage

We have made use of AI tools, mainly Claude and ChatGPT. Claude helped us read the existing code, write and edit code and configuration files (for example the Terraform files, help with deployment config in CI, and the READMEs , the grammar in this report). While using these tools we have reviewed the output ourselves, run it, and edited it when necessary. Everything else were solely handled by us.